

Memory Mapped IO

Theoretical background

Memory-Mapped I/O (MMIO) is a hardware–software interface technique where device registers are assigned fixed ranges in the system’s physical address space. The CPU uses normal load/store instructions to interact with these devices, making them appear like regular memory locations. Memory-mapped I/O uses the same address space to address both main memory and I/O devices. The memory and registers of the I/O devices are mapped to (associated with) address values, so a memory address may refer to either a portion of physical RAM or to memory and registers of the I/O device.

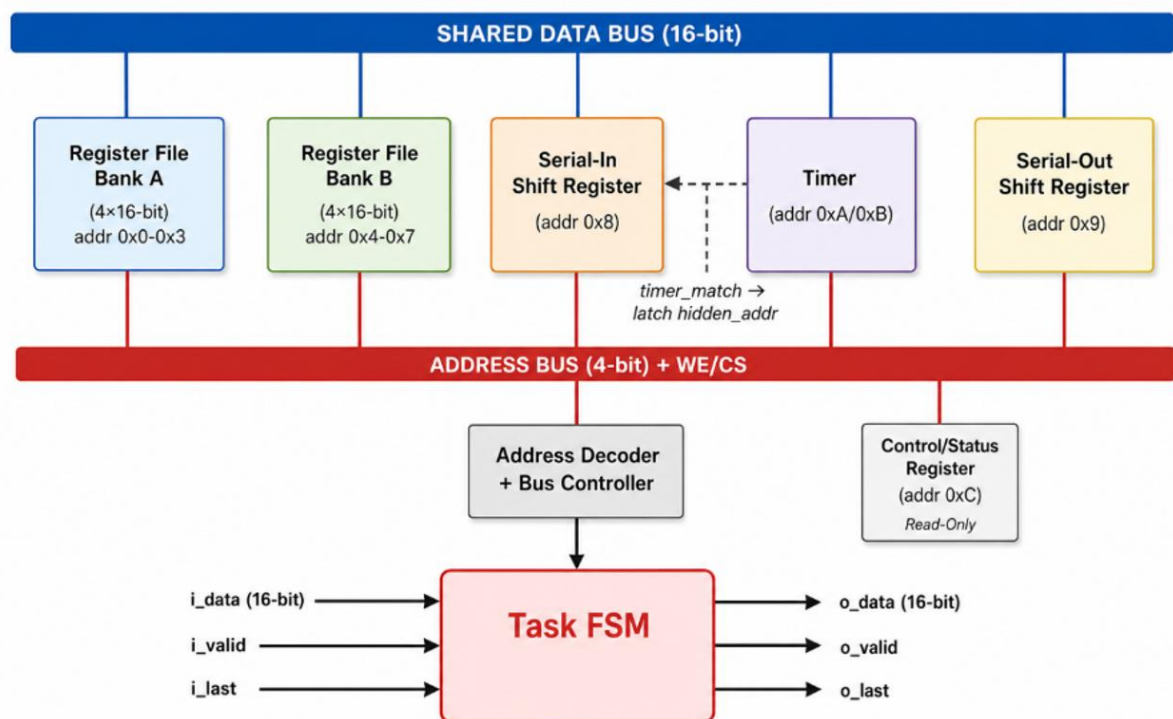


Figure 1 Memory-Mapped IO System Architecture (example).

Goal

Design a bus controller that interfaces with a memory-mapped system containing register files, shift registers, and a timer. Your module receives a stream of commands encoded as 16-bit samples. Each command instructs the controller to write data to peripherals, read from peripherals, or output a value. The challenge is to correctly decode the command protocol, drive the internal shared bus, and produce the correct output(s).

Specifically, the system should contain:

1. Register File Bank A (4×16 -bit entries) at addresses 0x0–0x3
2. Register File Bank B (4×16 -bit entries) at addresses 0x4–0x7
3. Serial-In Shift Register at address 0x8 (writing shifts data[0] into LSB)
4. Latch register at address 0x9 (writing loads parallel; parallel read)
5. Timer at addresses 0xA (write starts count from loaded value) and 0xB (write sets target)
6. Control/Status Register at address 0xC (read-only)

When the timer count reaches its target, the current value in the Serial-In Shift Register is latched as a hidden address. The participant must decode the input command stream, operate the peripherals in the correct order, and output the value stored at the hidden address.

Provided module

You can edit only the *task_10.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 1 List of inputs and outputs of the task_10 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	16 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	16 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet the task **always** receives 16 samples of input data, 16 bits each. The last sample in a packet is indicated by signal ***i_last***. The first sample is indicated by ***i_first***. The task should send **1 output sample**, 16 bits, and should indicate the last sample in the output packet by asserting ***o_last***.

Task receives data in the way shown in the figure below:

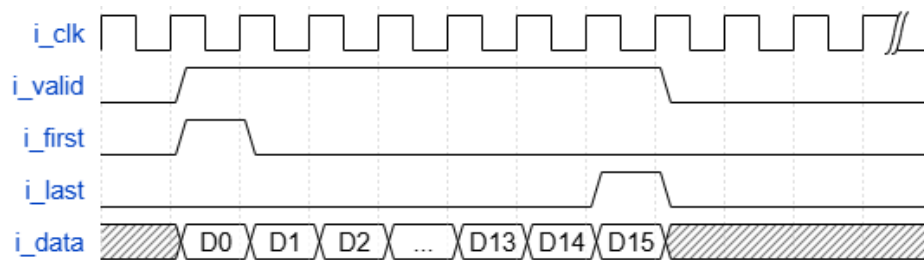


Figure 2 Input waveforms – the start and end of the packet.

To send data from the task in a proper way one needs to provide the ***o_valid*** signal that is synchronized with data on the ***o_data*** bus and ***o_last*** signal that is synchronized with the last sample in output packet.

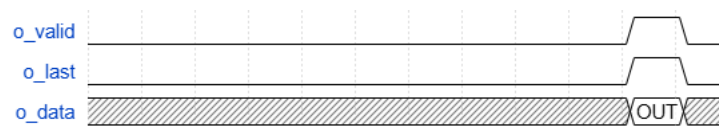


Figure 3 Output waveforms – the start and the end of the packet.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing ***o_valid*** and ***o_last*** signals.

Command encoding

Table 2 Command bit-field encoding.

Bits	Field	Description
[15:14]	Opcode	00=NOP, 01=WRITE, 10=READ, 11=OUTPUT
[13:10]	Reserved	Must be 0
[9:6]	Address	4-bit peripheral address (see memory map)
[5:0]	Reserved	Unused

- **NOP (00):** No operation. The controller does nothing.
- **WRITE (01):** The next input sample is treated as the 16-bit data to write at the given address.
- **READ (10):** Reads from the given address into an internal register (no output produced).

- **OUTPUT (11):** Reads from the given address and sends the result to o_data with o_valid=1. The last OUTPUT command in the packet must also have o_last=1.

Memory map

Table 3 Address space of the internal bus.

Address	Peripheral	Access	Description
0x0–0x3	Register File Bank A	R/W	4 general-purpose 16-bit registers
0x4–0x7	Register File Bank B	R/W	4 general-purpose 16-bit registers
0x8	Serial-In Shift Register	W	WRITE shifts wdata[0] into LSB; register shifts left
0x9	Serial-Out Latch Register	W/R	Timer triggers the 16-bit parallel write to it; parallel read
0xA	Timer Count	W	WRITE writes start value and enables timer
0xB	Timer Target	W	Sets the match target value
0xC	Control/Status	R	Status bits (see below)

Table 4 Control/Status register (address 0xC) bit fields.

Bits	Field	Description
[0]	timer_running	1 if timer is currently counting
[1]	timer_matched	1 if timer has reached target (latched)
[2]	secret_ready	1 if hidden address has been captured
[3]	serin_done	1 if ≥ 1 bits have been shifted into serial-in
[7:4]	hidden_addr	The latched address (captured from serial-in when timer fires)

Timer behaviour:

1. Writing to address 0xB sets the target value.
2. Writing to address 0xA loads the start count and begins counting (increments by 1 each clock cycle).

3. When `timer_count == timer_target`, the timer stops, `timer_matched` is set, and the current serial-in register value is latched as the hidden address.

The challenge

The input command stream will:

1. Write a secret value to one of the register file entries
2. Shift an address into the serial-in register (bit by bit)
3. Configure and start the timer
4. After the timer fires, the hidden address is revealed

Your task must correctly:

1. Decode all commands and drive the shared bus
2. Maintain the state of all peripherals
3. When an OUTPUT command arrives, read the correct address and produce the output

Evaluating the task

The task is evaluated by comparing your output against the expected secret value. The judge sends multiple test vectors with different secrets stored at different addresses. For each test:

- Correctness (pass/fail): `o_data` must exactly match the expected 16-bit secret value.
- The task is run 3 times per evaluation with different random configurations (different secret values, different register slots, different timer targets).
- There are 3 vectors, each awarding 5 points totaling 15 points.

Hints:

1. The FSM needs at least the states: IDLE, WAIT_FOR_WRITE_DATA, EXECUTE, OUTPUT.
2. The timer runs independently of your FSM: it counts every clock cycle once started.
3. The serial-in register shifts on every WRITE to address 0x8, not on every clock.
4. You do not need to implement the full serial-out for correctness: the OUTPUT command reads directly from the bus.
5. Use the control/status register (0xC) to programmatically discover the hidden address if needed.

5 additional points may be awarded for **resource utilization**. (15 points for 3 golden vectors test pass + 5 bonus points). For more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).